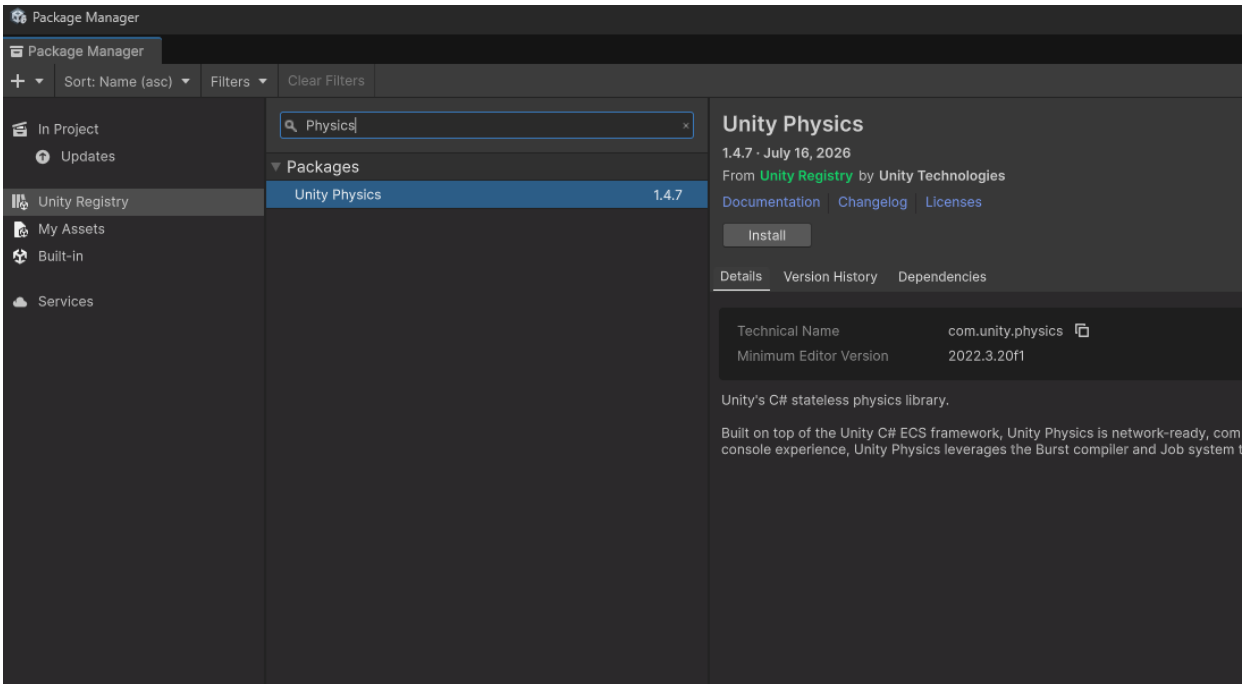
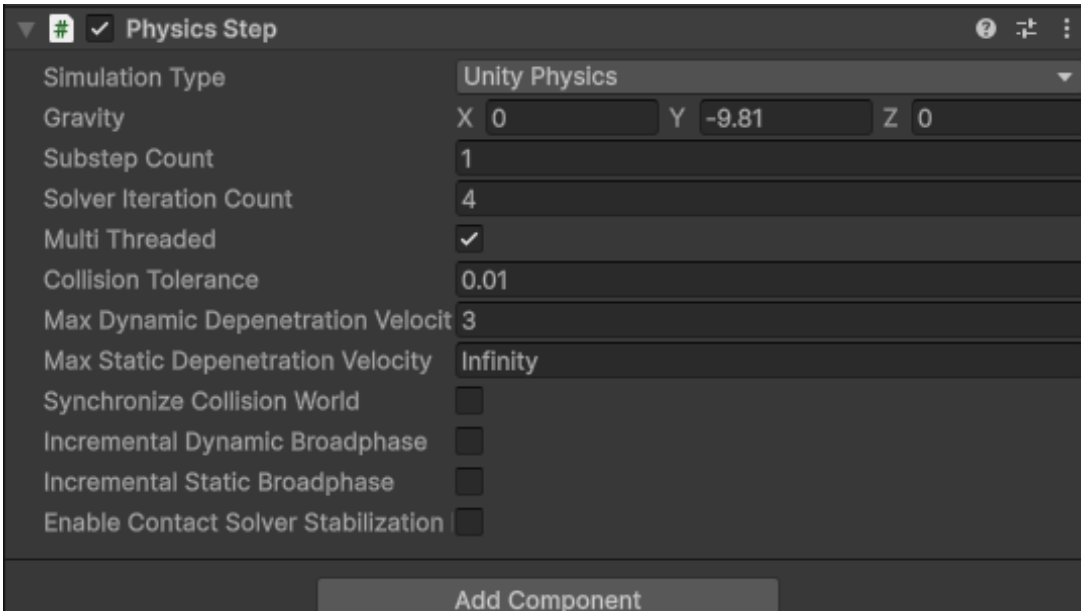
Для того, чтобы пользоваться физикой в DOTS, для начала необходимо установить Package “Unity Physics”.

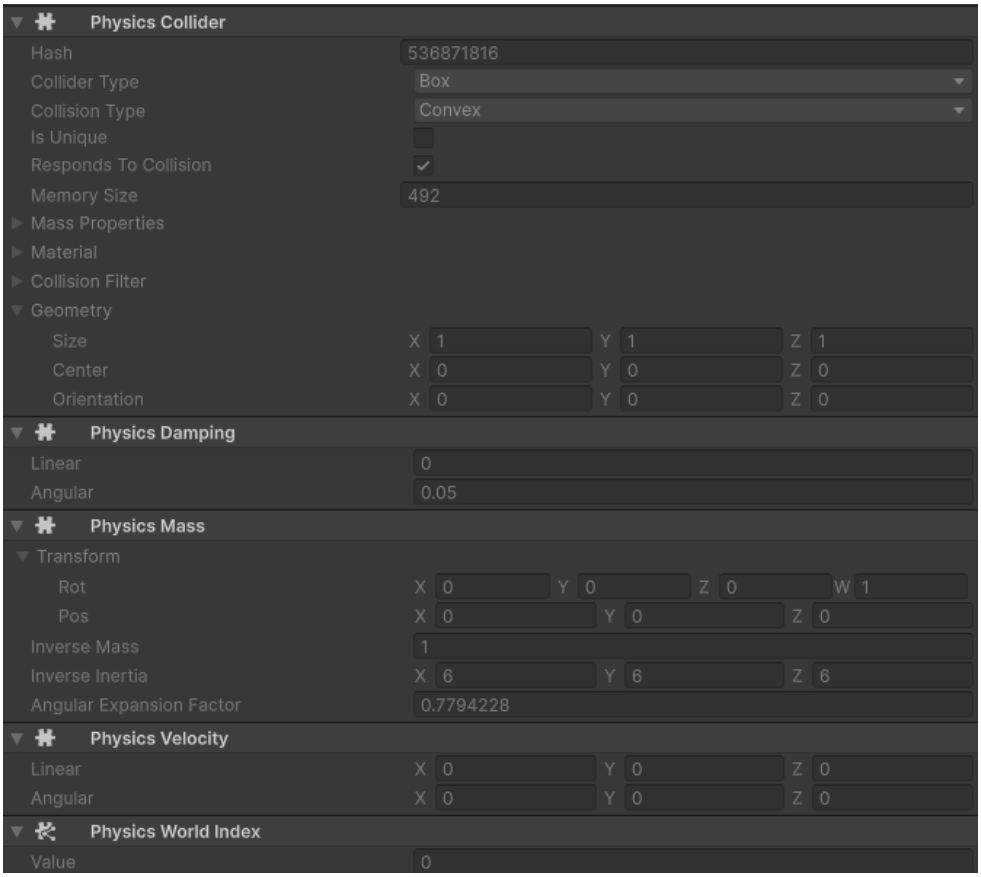


После установки Unity Physics, в сабсцену на синглтон необходимо добавить компонент «Physics Step». Он является конфигурацией физики в мире Entity.

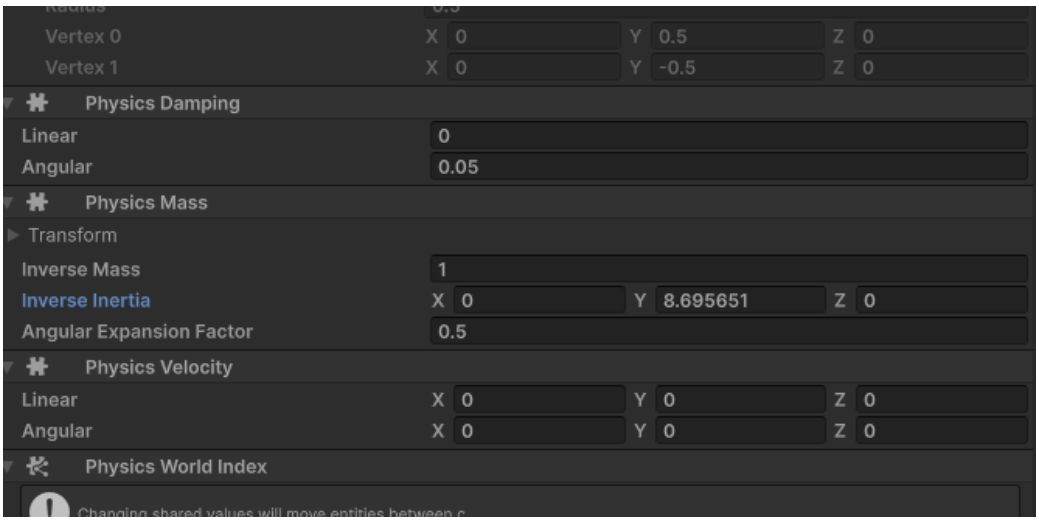


Теперь для того, чтобы сделать Entity физичным, на GameObject, из которого он запекается, необходимо повесить обычный Rigidbody и коллайдер. Если всё прошло успешно, в Runtime версии отображения данных появятся компоненты, отвечающие за физику. Их значения будут соответствовать заданным в Rigidbody и коллайдере. Неприятным исключением в этом

контексте являются Constraints на Rigidbody. На ECS компонентах нет их прямого аналога, который бы автоматически запекался.



Если нужно применить Rotation Constraints Rigidbody на некоторые оси, нужно вручную изменить параметр «Inverse Inertia» компонента Physics Mass. X Y и Z, выставленные в 0, ограничивают возможность физичного поворота объекта по соответствующим осям. То есть, если необходимо, чтобы объект находился в вертикальном положении, нужно выставить значения X и Z в 0.



Заставить объект двигаться и вращаться за счёт физики можно, меняя значения в компоненте Physics Velocity.

```
physicsVelocity.Linear = direction;

//float3 newPos = position + direction;

//localTransform.Position = newPos;
```

Логика слоёв коллайдеров в Unity Physics отличается от стандартной физики Unity. Что с чем будет и не будет сталкиваться задаётся не общей матрицей коллизий, а структурой CollisionFilter, которая есть у каждого коллайдера. Внутри этой структуры хранятся 3 значения: «Belongs To», «Collides With» и «Group Index».

Belongs To – слой, к которому принадлежит этот коллайдер. Является побитной маской (то есть, коллайдер может быть нескольких слоёв одновременно).

Collides With – это побитная маска, в которой хранятся слои, с которыми этот коллайдер сможет взаимодействовать.

Group Index – необычное значение. Если оно равно 0, то ни на что не влияет. Если же оно не равно 0, то влияет на коллайдеры с одинаковыми значениями. Если у двух коллайдеров совпадает значение Group Index, и оно больше 0, то эти два коллайдера могут взаимодействовать друг с другом вне зависимости от того, какие значения хранятся в Belongs To и Collides With. Если же значение совпадает и меньше 0, то эти два коллайдера не будут взаимодействовать независимо от значений Belongs To и Collides With.

```
CollisionFilter filter = new CollisionFilter()
{
    BelongsTo = 1 << 0, // этот коллайдер находится на 0 слое
    CollidesWith = (1 << 6) | (1 << 4), // он может взаимодействовать со слоями 6 и 4
    GroupIndex = 0 // группировка с другими коллайдерами по Group Index отключена
};
```

Для того, чтобы использовать методы, такие как CastRay, OverlapSphere итд внутри системы (как обычной, так и многопоточной версии), необходимо для начала получить CollisionWorld.

```
[BurstCompile]
Ссылок: 0
public void OnUpdate(ref SystemState state)
{
    PhysicsWorldSingleton physicsWorld = SystemAPI.GetSingleton<PhysicsWorldSingleton>();
    CollisionWorld collisionWorld = physicsWorld.CollisionWorld;
    var job = new TowerAttackTargetSelectJob()
    {
        CollisionWorld = collisionWorld
    };
    state.Dependency = job.ScheduleParallel(state.Dependency);
}
```

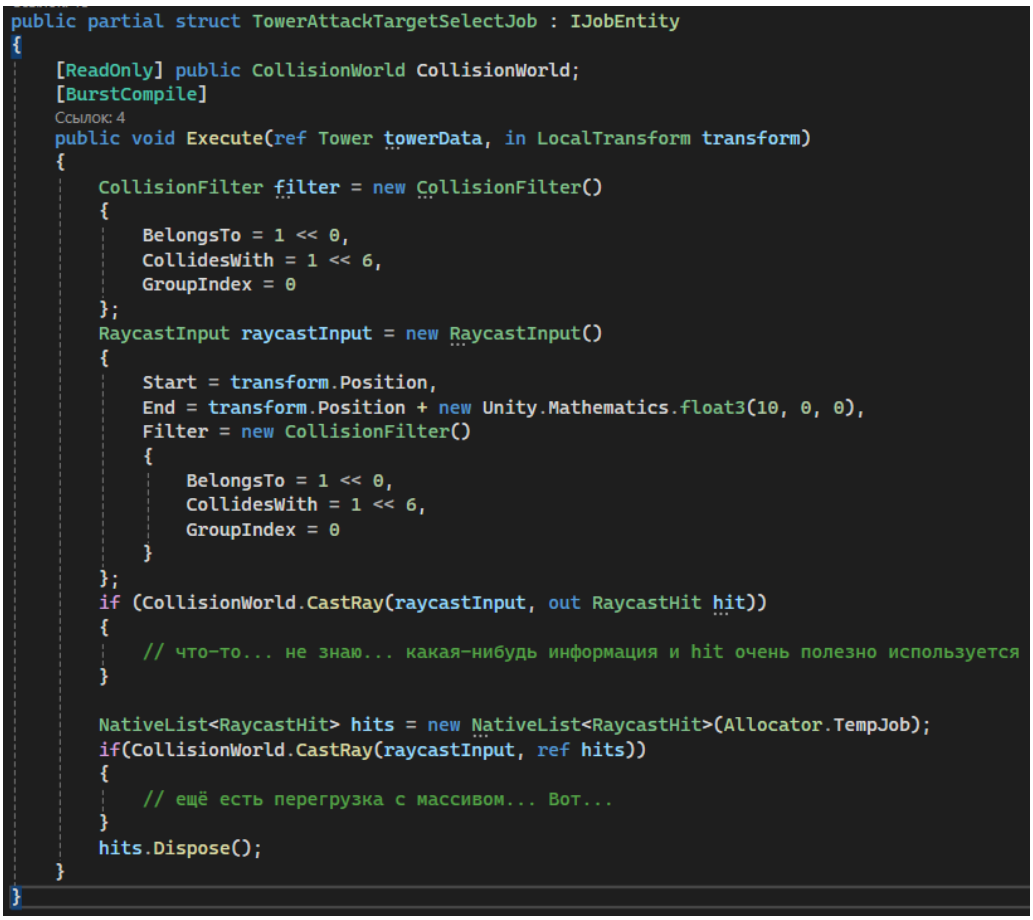
Далее примеры использования Raycast и OverlapSphere в джобе с использованием CollisionWorld:

```
public partial struct TowerAttackTargetSelectJob : IJobEntity
{
    [ReadOnly] public CollisionWorld CollisionWorld;
    [BurstCompile]
    Ссылок: 4
    public void Execute(ref Tower towerData, in LocalTransform transform)
    {
        NativeList<DistanceHit> hits = new NativeList<DistanceHit>(Allocator.TempJob);
        CollisionFilter filter = new CollisionFilter()
        {
            BelongsTo = 1 << 0,
            CollidesWith = 1 << 6,
            GroupIndex = 0
        };
        if(CollisionWorld.OverlapSphere(transform.Position, towerData.AttackRadius, ref hits, filter))
        {
            float minDist = float.PositiveInfinity;
            Entity foundEntity = Entity.Null;

            foreach(var hit in hits)
            {
                if(hit.Distance < minDist)
                {
                    minDist = hit.Distance;
                    foundEntity = hit.Entity;
                }
            }

            towerData.CurrentTarget = foundEntity;
        }
        hits.Dispose();

    }
}
```



```
public partial struct TowerAttackTargetSelectJob : IJobEntity
{
    [ReadOnly] public CollisionWorld CollisionWorld;
    [BurstCompile]
    Ссылок: 4
    public void Execute(ref Tower towerData, in LocalTransform transform)
    {
        CollisionFilter filter = new CollisionFilter()
        {
            BelongsTo = 1 << 0,
            CollidesWith = 1 << 6,
            GroupIndex = 0
        };
        RaycastInput raycastInput = new RaycastInput()
        {
            Start = transform.Position,
            End = transform.Position + new Unity.Mathematics.float3(10, 0, 0),
            Filter = new CollisionFilter()
            {
                BelongsTo = 1 << 0,
                CollidesWith = 1 << 6,
                GroupIndex = 0
            }
        };
        if (CollisionWorld.CastRay(raycastInput, out RaycastHit hit))
        {
            // что-то... не знаю... какая-нибудь информация и hit очень полезно используется
        }

        NativeList<RaycastHit> hits = new NativeList<RaycastHit>(Allocator.TempJob);
        if(CollisionWorld.CastRay(raycastInput, ref hits))
        {
            // ещё есть перегрузка с массивом... Вот...
        }
        hits.Dispose();
    }
}
```

Альтернативой OnCollision… и OnTrigger… методов в MonoBehaviour являются джобы ICollisionEventsJob и ITriggerEventsJob. Обе джобы требуют SimulationSingleton для своей работы. Также имеет смысл передавать в ICollisionEventsJob PhysicsWorld при необходимости, поскольку с его помощью можно получить из CollisionEvent дополнительные сведения о случившейся коллизии, например, точки контакта и импульс.

```
[UpdateInGroup(typeof(PhysicsSystemGroup))]
[UpdateAfter(typeof(PhysicsSimulationGroup))]
Ссылок: 1
partial struct PhysicsEventsSystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        var simulation = SystemAPI.GetSingleton<SimulationSingleton>();
        var physicsWorld = SystemAPI.GetSingleton<PhysicsWorldSingleton>().PhysicsWorld;

        var job = new CollisionEventJob()
        {
            physicsWorld = physicsWorld
        };

        state.Dependency = job.Schedule(simulation, state.Dependency);
    }
}

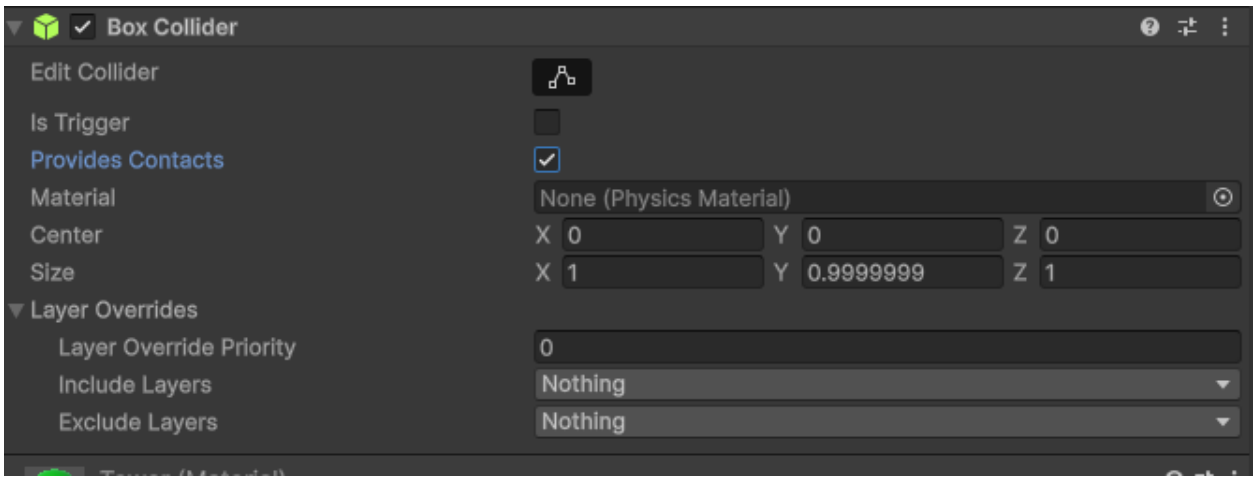

Ссылок: 2
public partial struct CollisionEventJob : ICollisionEventsJob
{
    [ReadOnly] public PhysicsWorld physicsWorld;
    Ссылок: 0
    public void Execute(CollisionEvent collisionEvent)
    {
        var entityA = collisionEvent.EntityA;
        var entityB = collisionEvent.EntityB;

        var details = collisionEvent.CalculateDetails(ref physicsWorld);

    }
}
```

```
Ссылок: 0
public partial struct TriggerEventJob : ITriggerEventsJob
{
    Ссылок: 0
    public void Execute(TriggerEvent triggerEvent)
    {
        var entityA = triggerEvent.EntityA;
        var entityB = triggerEvent.EntityB;
    }
}
```

Для того, чтобы эти ивенты начали воспринимать коллайдер и его коллизии, на коллайдере необходимо поставить «Provide Contacts» в значение true. Provide Contacts может быть включен только у одного объекта для того, чтобы коллизия этого объекта с другим была замечена ивентом.



Теперь, когда у коллайдеров включена эта опция, ICollisionEventsJob (или Trigger) проходит по всем случившимся с этими коллайдерами коллизиям в этот шаг симуляции физики. Разделения на Enter, Stay и Exit нет, джоба просто констатирует факт пересечения коллайдеров. При этом, что логично, среди всех случившихся коллизий могут быть как те, которые необходимо считывать, так и абсолютно бесполезные. Отсеивать полезные от бесполезных можно с помощью ComponentLookup. Также важно, что в CollisionEvent и TriggerEvent нет чёткой логики того, какой Entity окажется EntityA, а какой EntityB, а потому проверку с помощью ComponentLookup нужно проводить в обе стороны для надёжности.

```
public partial struct CollisionEventJob : ICollisionEventsJob
{
    [ReadOnly] public ComponentLookup<Tower> TowerLookup;
    [ReadOnly] public ComponentLookup<EnemyTag> EnemyLookup;
    public ComponentLookup<Health> HealthLookup;
    Ссылок: 0
    public void Execute(CollisionEvent collisionEvent)
    {
        var entityA = collisionEvent.EntityA;
        var entityB = collisionEvent.EntityB;

        if(TowerLookup.TryGetComponent(entityA, out _) && EnemyLookup.TryGetComponent(entityB, out _))
        {
            if(HealthLookup.TryGetRefRW(entityA, out RefRW<Health> towerHealth))
            {
                towerHealth.ValueRW.CurrentHealth -= 1;
            }
        }
        if (TowerLookup.TryGetComponent(entityB, out _) && EnemyLookup.TryGetComponent(entityA, out _))
        {
            if (HealthLookup.TryGetRefRW(entityB, out RefRW<Health> towerHealth))
            {
                towerHealth.ValueRW.CurrentHealth -= 1;
            }
        }
```

ICollisionEventsJob и ITriggerEventsJob – однопоточные джобы, поэтому бояться гонки данных с ними не нужно. Можно спокойно использовать ComponentLookup в том числе для записи.